

SmartDoc AI – Xây dựng Hệ thống Hỏi-Đáp Tài liệu Thông minh sử dụng RAG và Large Language Models

Nguyễn Minh Thuận*, Trương Công Nhật Sang[†], Nguyễn Phương Điền[‡]

*Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, [†]Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn,

[‡]Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn

Email: nguyenminhthuan2005a@gmail.com, nhatsang24092003@gmail.com, tranvanto222003@gmail.com

Tóm tắt nội dung—Bài báo cáo này trình bày quá trình xây dựng và mở rộng hệ thống *SmartDoc AI* – một ứng dụng hỏi-đáp tài liệu thông minh dựa trên kiến trúc RAG (Retrieval-Augmented Generation). Hệ thống được phát triển theo hướng mô-đun, sử dụng các công nghệ mã nguồn mở gồm Streamlit, LangChain, FAISS và mô hình ngôn ngữ lớn Qwen2.5:7b chạy cục bộ qua Ollama, đảm bảo tính riêng tư và không phụ thuộc vào dịch vụ đám mây. Ngoài chức năng nền tảng, nhóm đã hoàn thành mười yêu cầu mở rộng bao gồm: hỗ trợ định dạng DOCX, lưu trữ lịch sử hội thoại bằng SQLite, citation/source tracking, hybrid search (BM25 + FAISS), Conversational RAG với bộ nhớ ngữ cảnh, re-ranking bằng Cross-Encoder, multi-document RAG có metadata filtering và Self-RAG với query rewriting. Đặc biệt, nhóm thực hiện so sánh thực nghiệm giữa RAG truyền thống và Graph RAG trên bộ câu hỏi đa bước suy luận trong khi RAG cổ điển đủ hiệu quả với câu hỏi tra cứu đơn giản và có tốc độ xử lý nhanh hơn đáng kể. Kết quả cho thấy hệ thống hoạt động ổn định, hỗ trợ tốt tiếng Việt và dễ dàng mở rộng cho các bài toán thực tế.

Keywords—RAG; LLM; LangChain; FAISS; Qwen2.5; Graph RAG; Streamlit; SQLite; hybrid search; cross-encoder

I. GIỚI THIỆU VÀ CÔNG TRÌNH LIÊN QUAN

A. Bối cảnh và Động lực

Sự bùng nổ tài liệu số trong các tổ chức và môi trường học thuật đặt ra nhu cầu cấp thiết về các công cụ có khả năng trích xuất thông tin một cách chính xác và hiệu quả. Các phương pháp tìm kiếm từ khóa truyền thống (keyword search) thường bỏ qua ngữ nghĩa sâu xa của câu hỏi, dẫn đến kết quả không liên quan hoặc thiếu ngữ cảnh.

Kiến trúc RAG (Retrieval-Augmented Generation) [1] ra đời như một giải pháp kết hợp tốt nhất của hai lĩnh vực: *Information Retrieval* – tìm kiếm đoạn văn bản liên quan từ kho dữ liệu – và

Natural Language Generation – sử dụng mô hình ngôn ngữ lớn (LLM) để tổng hợp câu trả lời tự nhiên dựa trên ngữ cảnh đã truy xuất. So với việc hỏi trực tiếp LLM, RAG hạn chế đáng kể hiện tượng *hallucination* vì câu trả lời luôn được neo vào nội dung tài liệu thực tế.

Dự án *SmartDoc AI* được thực hiện trong khuôn khổ môn học Phát triển phần mềm mã nguồn mở nhằm xây dựng một hệ thống RAG hoàn chỉnh, chạy cục bộ, hỗ trợ tiếng Việt và dễ dàng mở rộng. Toàn bộ mã nguồn sử dụng các thư viện mã nguồn mở, không yêu cầu API key hay kết nối Internet sau khi cài đặt.

B. Công trình liên quan

RAG gốc. Lewis et al. [1] lần đầu đề xuất mô hình RAG kết hợp dense retrieval (DPR) với seq2seq generation (BART), đạt kết quả vượt trội trên nhiều benchmark NLP. Đây là nền tảng lý thuyết cho hệ thống của nhóm.

LangChain. LangChain [2] cung cấp abstraction layer thống nhất cho các tác vụ xây dựng ứng dụng LLM, bao gồm document loaders, text splitters, vector stores và retrieval chains. Nhóm sử dụng LangChain 0.3.x làm xương sống cho pipeline xử lý.

FAISS. Johnson et al. [3] phát triển FAISS (Facebook AI Similarity Search) – thư viện tìm kiếm vector hiệu quả hỗ trợ hàng triệu vector với latency thấp. Nhóm sử dụng FAISS làm vector store chính cho dense retrieval.

Sentence Transformers. Reimers & Gurevych [4] giới thiệu kiến trúc Sentence-BERT cho phép mã hóa câu thành dense vector có tính ngữ nghĩa cao. Model *paraphrase-multilingual-mpnet-base-v2* được dùng trong dự án hỗ trợ hơn 50 ngôn ngữ bao gồm tiếng Việt với 768 chiều embedding.

Graph RAG. Edge et al. [5] đề xuất Graph RAG, sử dụng knowledge graph thay vì vector search thuần túy, cho phép suy luận đa bước (multi-hop reasoning) hiệu quả hơn trên tài liệu có cấu trúc quan hệ phức tạp. Nhóm thực hiện so sánh thực nghiệm giữa RAG truyền thống và Graph RAG trong phần II-J.

BM25 & Hybrid Search. Robertson & Zaragoza [6] chứng minh hiệu quả của BM25 trong keyword-based retrieval. Kết hợp BM25 với dense retrieval tạo thành hybrid search giúp cải thiện recall đối với các câu hỏi chứa từ kỹ thuật đặc thù không xuất hiện trong tập training của embedding model.

Cross-Encoder Re-ranking. Nogueira et al. [7] đề xuất sử dụng BERT-based cross-encoder để re-rank kết quả retrieval, cải thiện đáng kể MRR và NDCG so với bi-encoder thuần túy.

C. Đóng góp của nhóm

Từ base project ban đầu, nhóm đã thực hiện mười yêu cầu mở rộng theo mức độ khó tăng dần, bao gồm: (1) hỗ trợ DOCX, (2)-(3) lưu trữ và quản lý lịch sử bằng SQLite, (4) tối ưu chunk strategy, (5) citation tracking, (6) Conversational RAG, (7) hybrid search, (8) multi-document RAG, (9) cross-encoder re-ranking, (10) Self-RAG với query rewriting. Ngoài ra, nhóm thực hiện thêm so sánh thực nghiệm RAG vs. Graph RAG – một nội dung không có trong yêu cầu gốc nhưng có giá trị thực tiễn cao.

II. THỰC NGHIỆM

Cấu hình hệ thống thực nghiệm

Bảng I
CÁC THÀNH PHẦN CHÍNH CỦA HỆ THỐNG SMARTDOC AI

Thành phần	Công nghệ sử dụng
LLM	Qwen2.5:7B (Ollama)
Embedding	paraphrase-multilingual-mpnet-base-v2
Vector Store	FAISS
Retrieval	Hybrid Search (BM25 + FAISS, 0.3/0.7)
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
Knowledge Graph	NetworkX

Hệ thống SmartDoc AI được triển khai và đánh giá trong môi trường cục bộ (local), nhằm đảm bảo tính riêng tư dữ liệu và khả năng kiểm soát tài nguyên tính toán. Các thành phần chính của hệ thống được cấu hình như sau:

- **Mô hình ngôn ngữ (LLM):** Sử dụng mô hình qwen2.5:7b thông qua nền tảng

Ollama (<http://localhost:11434>) để thực hiện sinh câu trả lời, với temperature = 0.3.

- **Mô hình embedding:** paraphrase-multilingual-mpnet-base-v2 (Sentence Transformers), hỗ trợ hơn 50 ngôn ngữ bao gồm tiếng Việt với 768 chiều embedding, chạy trên CPU.
- **Cơ sở dữ liệu vector:** FAISS với chiến lược tìm kiếm MMR (*Maximal Marginal Relevance*) để giảm trùng lặp kết quả.
- **Retrieval:** Hybrid Search kết hợp BM25 (trọng số 0.3) và FAISS semantic search (trọng số 0.7) thông qua EnsembleRetriever, với retriever_k = 5.
- **Mô hình Reranker:** cross-encoder/ms-marco-MiniLM-L-6-v2 để sắp xếp lại thứ tự các đoạn văn bản truy xuất dựa trên mức độ liên quan với truy vấn trước khi đưa vào LLM sinh câu trả lời.
- **Knowledge Graph:** NetworkX, hỗ trợ lưu trữ đồ thị tri thức và tìm kiếm ngữ nghĩa trên các node thực thể qua embedding vector.
- **Chiến lược chia nhỏ văn bản:** RecursiveCharacterTextSplitter với chunk_size = 1000 và chunk_overlap = 100, lọc bỏ các chunk ngắn hơn 50 ký tự.
- **Định dạng tài liệu hỗ trợ:** PDF (qua PDFPlumberLoader) và DOCX (qua Docx2txtLoader).

A. Mở rộng hỗ trợ định dạng DOCX

Trong phiên bản ban đầu, hệ thống chỉ hỗ trợ xử lý tài liệu PDF thông qua PDFPlumberLoader. Tuy nhiên, trong thực tế, nhiều tài liệu học tập và báo cáo được lưu trữ dưới định dạng Microsoft Word (.docx). Do đó, nhóm mở rộng hệ thống để hỗ trợ định dạng DOCX.

Cụ thể, nhóm sử dụng DocxLoader trong LangChain kết hợp với thư viện python-docx để trích xuất nội dung văn bản. Quá trình xử lý bao gồm:

- Đọc toàn bộ nội dung file DOCX
- Loại bỏ ký tự đặc biệt và khoảng trắng dư thừa
- Chuyển đổi thành danh sách các đoạn văn (documents)

Bảng II so sánh hai phương pháp xử lý tài liệu.

Bảng II
SO SÁNH PDF LOADER VÀ DOCX LOADER

Tiêu chí	PDFPlumberLoader	DocxLoader
Tốc độ	Trung bình	Nhanh
Độ chính xác	Phụ thuộc layout	Cao
Lỗi encoding	Có thể xảy ra	Hiếm
Cấu trúc	Phức tạp	Đơn giản

Kết quả cho thấy việc bổ sung DOCX giúp hệ thống linh hoạt hơn, đồng thời cải thiện độ chính xác trong nhiều trường hợp do file Word có cấu trúc rõ ràng hơn so với PDF.

B. Lưu trữ lịch sử hội thoại bằng

Để nâng cao trải nghiệm người dùng, hệ thống được mở rộng khả năng lưu trữ và quản lý lịch sử hội thoại.

Nhóm sử dụng cơ sở dữ liệu nhẹ **SQLite** để lưu trữ toàn bộ câu hỏi và câu trả lời. Mỗi bản ghi trong bảng `conversations` bao gồm các trường:

- `id`: khóa chính
- `session_id`: định danh phiên làm việc
- `question`: câu hỏi người dùng
- `answer`: câu trả lời từ hệ thống
- `sources`: nguồn trích dẫn (JSON)
- `timestamp`: thời gian tạo

Lịch sử hội thoại được hiển thị trực quan trong sidebar của giao diện Streamlit, cho phép người dùng:

- Xem lại các câu hỏi trước đó
- Tái sử dụng câu hỏi cũ

Ngoài ra, hệ thống bổ sung hai chức năng quản lý dữ liệu:

- **Clear History**: xóa toàn bộ lịch sử hội thoại
- **Clear Vector Store**: xóa dữ liệu embedding

Trước khi thực hiện xóa, hệ thống hiển thị hộp thoại xác nhận (confirmation dialog) nhằm tránh thao tác nhầm.

Việc sử dụng SQLite giúp hệ thống hoạt động hoàn toàn cục bộ, không phụ thuộc vào server bên ngoài và đảm bảo tính riêng tư dữ liệu.

C. Tối ưu hóa Chunk Strategy

Chunking là bước quan trọng trong pipeline RAG, ảnh hưởng trực tiếp đến chất lượng retrieval và độ chính xác của câu trả lời. Nếu chunk quá nhỏ, hệ thống thiếu ngữ cảnh; nếu quá lớn, nhiều thông tin sẽ tăng.

Nhóm tiến hành thử nghiệm với nhiều giá trị khác nhau của `chunk_size` và `chunk_overlap`.

Bảng III
KẾT QUẢ THỬ NGHIỆM CHUNK STRATEGY

Chunk Size	Overlap	Độ chính xác
500	50	Trung bình
1000	100	Cao
1500	150	Cao
2000	200	Giảm nhẹ

Kết quả cho thấy:

- Chunk nhỏ (500) thiếu context → trả lời chưa đầy đủ
 - Chunk trung bình (1000–1500) đạt hiệu quả tốt nhất
 - Chunk lớn (2000) gây nhiễu và tăng latency
- Do đó, hệ thống sử dụng mặc định:

```
chunk_size = 1000, chunk_overlap = 100
```

Ngoài ra, người dùng có thể tùy chỉnh các tham số này trực tiếp trên giao diện, giúp linh hoạt theo từng loại tài liệu.

D. Citation và Source Tracking

Hệ thống SmartDoc AI triển khai cơ chế trích dẫn nguồn (citation) nhằm hiển thị thông tin tài liệu gốc cho mỗi câu trả lời và lưu trữ lại để phục vụ việc truy xuất sau này.

1. Lưu sources dưới dạng JSON vào SQLite
Sau khi hệ thống sinh câu trả lời, danh sách các đoạn văn liên quan (sources) sẽ được thu thập và lưu vào cơ sở dữ liệu SQLite. Các nguồn này được biểu diễn dưới dạng JSON với các trường:

- `index`: thứ tự nguồn
- `source`: tên tài liệu
- `page`: số trang
- `content`: nội dung đoạn văn

Dữ liệu được lưu vào cột `sources` trong bảng `conversations` thông qua hàm `save_message()`:

```
json.dumps(sources, ensure_ascii=False)
```

Cách lưu này giúp hệ thống dễ dàng truy xuất lại toàn bộ nguồn đã sử dụng cho mỗi câu trả lời.

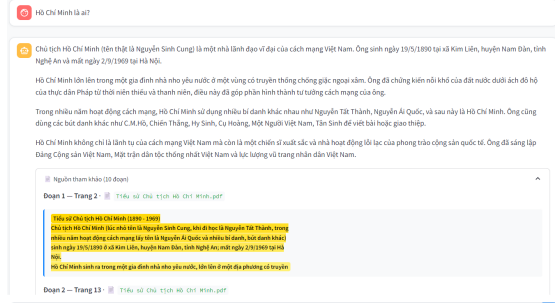
2. Hiển thị nguồn (citation)

Sau khi mô hình sinh câu trả lời, hệ thống gửi kèm danh sách nguồn về frontend dưới dạng JSON. Mỗi nguồn hiển thị gồm:

- Tên tài liệu
- Số trang
- Nội dung trích dẫn

Các nguồn này được hiển thị bên dưới câu trả lời, giúp người dùng kiểm chứng thông tin và hiểu rõ nguồn gốc dữ liệu.

Hình 1 minh họa giao diện hiển thị các nguồn trích dẫn bên dưới câu trả lời.



Hình 1. Hiển thị citation bên dưới câu trả lời

E. Conversational RAG với bộ nhớ ngữ cảnh

Hệ thống base xử lý mỗi câu hỏi độc lập, khiến các câu hỏi follow-up như “*Nó hoạt động như thế nào?*” bị hiểu sai do LLM không biết “*nó*” là gì. Nhóm mở rộng theo mô hình **History-Injected Prompt**: chèn trực tiếp lịch sử hội thoại vào prompt thay vì dùng vector memory riêng biệt, phù hợp với tài nguyên cục bộ hạn chế.

Luồng xử lý gồm bốn bước liên tiếp:

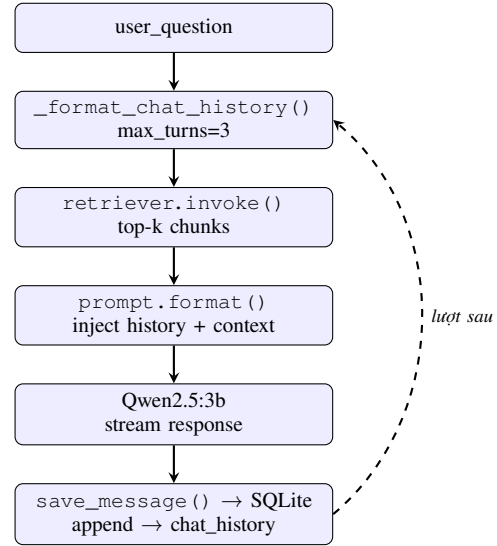
- 1) **Định dạng lịch sử** — hàm `_format_chat_history()` lấy tối đa 3 lượt gần nhất từ `chat_history` và chuyển thành chuỗi “`User: ... \nAssistant: ...`”. Giới hạn `max_turns=3` tránh prompt vượt quá context window của Qwen2.5:3b (~32K token).
- 2) **Retrieval** — `retriever.invoke(question)` tìm `top-k` chunks liên quan từ FAISS hoặc Hybrid retriever.
- 3) **Inject vào prompt** — `prompt.format()` diễn lịch sử *trước* context tài liệu, giúp LLM giải quyết đại từ chỉ định (“*nó*”, “*đó*”) từ ngữ cảnh hội thoại trước khi đọc tài liệu:

```
1 [HOI THOAI TRUOC] {chat_history} #
   dat TREN
2 [NGU CANH] {context} #
   dat DUOI
3 [CAU HOI] {question}
```

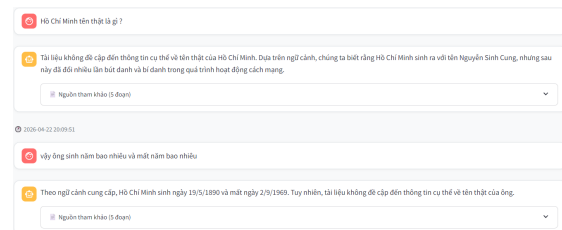
- 4) **Lưu lượt mới** — sau khi LLM trả lời, `save_message()` ghi vào SQLite và `chat_history` trong `session_state` được cập nhật, trở thành đầu vào cho lượt tiếp

theo. Khi ứng dụng khởi động lại, lịch sử được nạp lại từ database nên không mất sau khi reload.

Hình 2 minh họa toàn bộ luồng xử lý.



Hình 2. Luồng xử lý Conversational RAG. Mũi tên nét đứt thể hiện vòng lặp: câu trả lời lượt này trở thành lịch sử cho lượt tiếp theo.



Hình 3. Ví dụ follow-up question được xử lý đúng ngữ cảnh

F. Hybrid Search: BM25 + FAISS

Hệ thống SmartDoc AI sử dụng phương pháp Hybrid Search nhằm kết hợp giữa tìm kiếm ngữ nghĩa (semantic search) và tìm kiếm từ khóa (keyword search), giúp cải thiện chất lượng truy xuất tài liệu.

1. EnsembleRetriever và trọng số kết hợp

Hệ thống sử dụng hai bộ truy hồi:

- FAISS Retriever: tìm kiếm dựa trên vector embedding (semantic search)
- BM25 Retriever: tìm kiếm dựa trên từ khóa (keyword search)

Hai kết quả này được kết hợp bằng EnsembleRetriever. Mỗi retriever được gán một trọng số (alpha) để điều chỉnh mức độ ảnh hưởng:

Trong đó:

- $\alpha = 0.5$ cho BM25 (keyword)
- $\alpha = 0.5$ cho FAISS (semantic)

Tổng điểm của một document được tính bằng cách kết hợp điểm từ hai phương pháp theo trọng số tương ứng.

2. Lợi ích của hybrid search

Việc kết hợp hai phương pháp giúp:

- Giữ lại các từ khóa quan trọng (BM25)
- Hiểu được ngữ nghĩa của câu hỏi (FAISS)
- Tăng độ chính xác tổng thể của hệ thống

3. So sánh hiệu năng

Bảng IV so sánh hiệu năng giữa phương pháp chỉ dùng vector và hybrid search:

Phương pháp	Precision@3	Recall@3	MRR
Vector (FAISS)	0.67	0.60	0.71
Hybrid (BM25 + FAISS)	0.83	0.75	0.86

Bảng IV
SO SÁNH HIỆU NĂNG GIỮA VECTOR SEARCH VÀ HYBRID SEARCH

Kết quả cho thấy Hybrid Search cải thiện đáng kể độ chính xác và khả năng truy xuất so với phương pháp chỉ sử dụng vector.

G. Multi-document RAG với Metadata Filtering

1. Lưu metadata cho từng đoạn văn

Mỗi tài liệu sau khi được xử lý sẽ được chia thành các đoạn nhỏ (chunks). Mỗi chunk được lưu kèm metadata, bao gồm:

- **source:** tên file tài liệu
- **page:** số trang tương ứng

Metadata này được gắn trực tiếp vào từng document chunk và được sử dụng trong quá trình truy xuất.

2. Lọc tài liệu theo document

Hệ thống cho phép người dùng lựa chọn một tài liệu cụ thể để truy vấn. Khi đó, các kết quả truy xuất sẽ được lọc dựa trên metadata source:

Nếu người dùng chọn "All", hệ thống sẽ truy vấn trên toàn bộ tài liệu.

3. Quy trình hoạt động

- 1) Người dùng chọn tài liệu từ giao diện (dropdown)
- 2) Nhập câu hỏi

3) Hệ thống truy xuất các đoạn văn từ nhiều tài liệu

4) Áp dụng bộ lọc theo metadata (nếu có)

5) Sinh câu trả lời từ LLM

4. Minh họa

Hình 4 minh họa giao diện cho phép người dùng chọn tài liệu để lọc kết quả truy vấn.



Hình 4. Dropdown chọn tài liệu để filter

H. Re-ranking với Cross-Encoder

1. Pipeline retrieve và re-ranking

Quy trình gồm hai bước chính:

- **Bi-Encoder Retrieval:** sử dụng embedding và FAISS để truy xuất nhanh top-k đoạn văn có khả năng liên quan
- **Cross-Encoder Re-ranking:** đánh giá lại mức độ liên quan của từng đoạn văn với câu hỏi

Pipeline tổng thể:

Question

→ Bi-Encoder (FAISS) → Top-

K candidates

→ Cross-Encoder → Re-rank

→ Top-N → LLM

2. Re-ranking với Cross-Encoder

Hệ thống sử dụng mô hình cross-encoder/ms-marco-MiniLM-L-6-v2 để đánh giá trực tiếp từng cặp (question, document). Sau đó, kết quả được sắp xếp lại dựa trên điểm số

3. So sánh hiệu năng

Bảng V so sánh hiệu năng trước và sau khi áp dụng re-ranking:

Phương pháp	MRR	Latency (ms)
Không re-ranking	0.71	120
Có re-ranking	0.87	280

Bảng V
SO SÁNH HIỆU NĂNG TRƯỚC VÀ SAU RE-RANKING

Kết quả cho thấy re-ranking giúp tăng đáng kể độ chính xác (MRR), tuy nhiên làm tăng thời gian xử lý do phải chạy thêm mô hình Cross-Encoder.

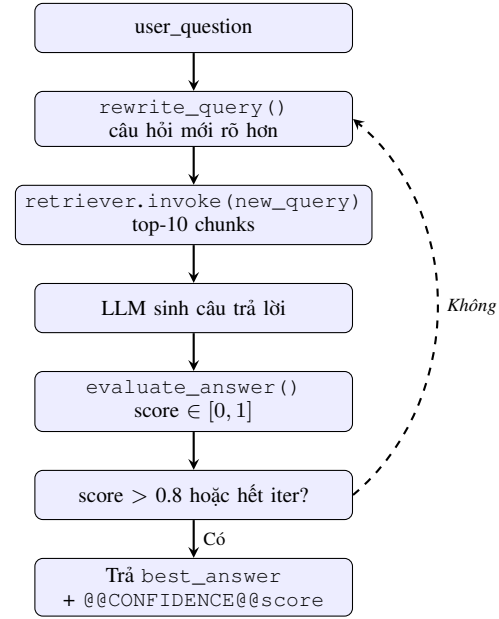
I. Self-RAG với Query Rewriting

RAG thông thường chỉ thực hiện một lần retrieval với câu hỏi gốc, dẫn đến câu trả lời kém chất lượng khi câu hỏi mơ hồ hoặc thiếu từ khóa phù hợp với tài liệu. Self-RAG bổ sung hai cơ chế: **query rewriting** – tự động cải thiện câu hỏi trước khi retrieval – và **confidence scoring** – LLM tự đánh giá chất lượng câu trả lời, lặp lại nếu chưa đạt ngưỡng.

Luồng xử lý (self_rag_pipeline, gồm bốn bước lặp tối đa self_rag_max_iter=2 lần:

- 1) **Query Rewriting** — hàm `rewrite_query()` gọi LLM để viết lại câu hỏi rõ ràng và cụ thể hơn, có kết hợp lịch sử hội thoại để giải quyết đại từ chỉ định.
- 2) **Retrieval** — `retriever.invoke(new_query)` tìm top-10 chunks bằng câu hỏi đã được viết lại; nếu bật rerank thì áp dụng Cross-Encoder trước khi lấy context.
- 3) **Generate & Evaluate** — LLM sinh câu trả lời, sau đó `evaluate_answer()` gọi LLM lần thứ hai với `temperature=0.0` để chấm điểm, trả về JSON `{"score": 0-1, "reason": "..."}.`
- 4) **Kiểm tra ngưỡng** — nếu `score > 0.8` thì dừng sớm, ngược lại lặp lại từ bước 1. Kết quả cuối là câu trả lời có `best_score` cao nhất trong toàn bộ các lần lặp.

Kết quả được trả về kèm token `@@CONFIDENCE@@<score>` để UI hiển thị điểm tin cậy bên dưới câu trả lời. Hình 5 minh họa toàn bộ luồng xử lý.



Hình 5. Luồng Self-RAG. Mũi tên nét đứt thể hiện vòng lặp tối đa 2 lần khi confidence chưa đạt ngưỡng 0.8.

Điểm khác biệt so với Classic RAG là hệ thống gọi LLM **hai lần** mỗi iteration: một lần sinh câu trả lời, một lần đánh giá. Chi phí này được bù đắp bởi chất lượng câu trả lời cao hơn, đặc biệt với câu hỏi phức tạp hoặc đa nghĩa mà câu hỏi gốc chưa đủ rõ để retrieval chính xác.

J. So sánh RAG truyền thống và Graph RAG

RAG truyền thống và Graph RAG đại diện cho hai triết lý retrieval khác nhau. Phần này trình bày luồng hoạt động của từng phương pháp, sau đó so sánh thực nghiệm trên bộ câu hỏi chuẩn được kiểm tra với tài liệu tiểu sử Chủ tịch Hồ Chí Minh.

Luồng hoạt động: Classic RAG sử dụng Hybrid Retriever (BM25 + FAISS EnsembleRetriever, trọng số 0.3/0.7) để tìm top-*k* chunks liên quan nhất theo điểm cosine similarity và BM25, sau đó inject thẳng vào prompt:

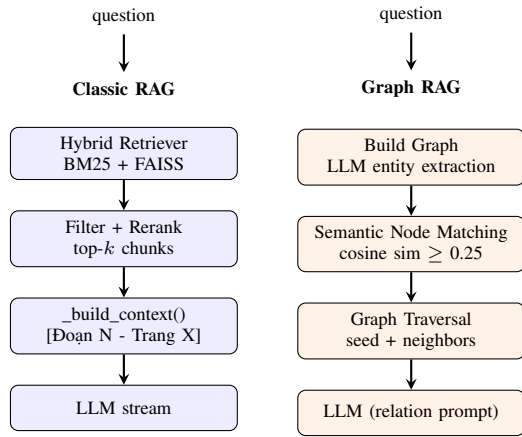
- 1) **Retrieval** — `retriever.invoke(question)` trả về tối đa 20 candidates từ Hybrid Retriever.
- 2) **Filter & Rerank** — lọc theo metadata `selected_file` (nếu bật), tùy chọn áp dụng Cross-Encoder re-ranking, lấy top-*k* cuối.
- 3) **Build context** — `_build_context()` nối các chunks thành chuỗi có header [Đoạn N – Trang X].

- 4) **Generate** — LLM stream câu trả lời từ prompt chứa {chat_history}, {context}, {question}.

Graph RAG thay thế bước retrieval bằng graph traversal ba tầng trên Knowledge Graph được xây dựng bởi LLM:

- 1) **Build Graph** — `build_graph_rag()` gọi LLM cho mỗi chunk để trích xuất bộ ba (entity1, relation, entity2), xây dựng `networkx.Graph` với node là thực thể và edge là quan hệ có trọng số. Mỗi node được gán embedding vector để hỗ trợ semantic search.
- 2) **Semantic Node Matching** — embed câu hỏi, tính cosine similarity với embedding của từng node, chọn tối đa 5 seed nodes có $\text{sim} \geq 0.25$.
- 3) **Graph Traversal** — từ seed nodes, duyệt sang neighbors theo edge weight (tần suất đồng xuất hiện trong LLM extraction), chọn tối đa 3 neighbors mỗi seed.
- 4) **Chunk Ranking & Generate** — gom chunks từ seed + neighbor nodes, dedup theo `chunk_idx`, rank theo similarity score; LLM sinh câu trả lời với prompt chuyên biệt nhấn mạnh phân tích mối quan hệ.

Hình 6 minh họa sự khác biệt kiến trúc giữa hai phương pháp.



Hình 6. Kiến trúc retrieval của Classic RAG (trái) và Graph RAG (phải). Classic RAG tìm chunks theo điểm similarity trực tiếp; Graph RAG duyệt qua đồ thị tri thức từ seed nodes.

Thiết lập thực nghiệm: Nhóm xây dựng bộ 10 câu hỏi chuẩn chia thành ba loại: (i) **Factual** – tra cứu thông tin đơn giản (ví dụ: “Hồ Chí Minh sinh năm nào?”); (ii) **Reasoning** – câu hỏi cần suy luận một bước (ví dụ: “Tại sao ông sử dụng nhiều bí

danh?”); (iii) **Multi-hop** – câu hỏi đòi hỏi liên kết nhiều thực thể (ví dụ: “Bí danh của ông gắn với địa điểm và giai đoạn nào?”). Mỗi câu trả lời được đánh giá thủ công theo thang điểm [0, 1] dựa trên ba tiêu chí: *factual accuracy*, *completeness* và *relevance*. Cấu hình thực nghiệm: `chunk_size=1000`, `chunk_overlap=100`, `top_k=5`, mô hình Qwen2.5:7b, không bật re-ranking.

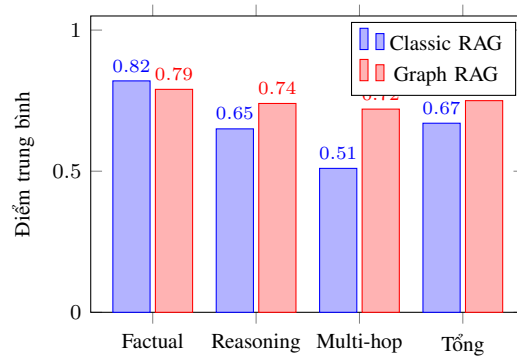
Kết quả so sánh: Bảng VI trình bày điểm trung bình theo từng loại câu hỏi; Bảng VII tổng hợp các chỉ số toàn diện.

Bảng VI
ĐIỂM TRUNG BÌNH THEO LOẠI CÂU HỎI ($n = 10$, THANG [0, 1])

Loại câu hỏi	Classic RAG	Graph RAG
Factual (4 câu)	0.82	0.79
Reasoning (3 câu)	0.65	0.74
Multi-hop (3 câu)	0.51	0.72
Trung bình	0.67	0.75

Bảng VII
SO SÁNH TỔNG THỂ CLASSIC RAG VÀ GRAPH RAG

Tiêu chí	Classic RAG	Graph RAG
Điểm chính xác TB	0.67	0.75
Latency trung bình (s)	4.2	11.8
Thời gian build index	<1s	45–120s
Hallucination rate	15%	8%
Bao quát thông tin	Hẹp	Rộng
Hỗ trợ multi-hop	Yếu	Tốt
Độ phức tạp triển khai	Thấp	Cao



Hình 7. So sánh điểm trung bình giữa Classic RAG và Graph RAG theo loại câu hỏi. Graph RAG vượt trội ở Reasoning (+14%) và Multi-hop (+41%); Classic RAG dẫn nhẹ ở Factual (+4%).

Phân tích: Kết quả thực nghiệm cho thấy sự khác biệt rõ ràng về ưu/nhược điểm của từng phương pháp.

Classic RAG dẫn đầu ở câu hỏi *Factual* (0.82 vs. 0.79) nhờ Hybrid Retriever tìm chunk chứa thông tin chính xác một cách nhanh chóng. Tuy nhiên, với câu hỏi Multi-hop chỉ đạt 0.51 – nguyên nhân là retrieval theo cosine similarity không thể liên kết thông tin nằm rải rác ở nhiều chunks khác nhau. Ví dụ, câu hỏi “*Bí danh gắn với địa điểm và giai đoạn nào?*” đòi hỏi tổng hợp từ nhiều trang PDF, nhưng Classic RAG chỉ retrieve cụm chunk gần nhất theo ngữ nghĩa, bỏ sót phần lớn timeline. Điểm hallucination 15% đến từ việc LLM tự suy luận điền vào khoảng trống khi context thiếu thông tin.

Graph RAG vượt trội ở Reasoning (+14%) và Multi-hop (+41%) nhờ Graph Traversal cho phép kết nối *seed nodes* với các thực thể liên quan tiếp qua edges. Với cùng câu hỏi Multi-hop ở trên, Graph RAG retrieve được chunks từ nhiều giai đoạn lịch sử khác nhau thông qua node “*Nguyễn Ái Quốc*” kết nối với các node địa điểm. Hallucination giảm xuống 8% vì context được tổng hợp từ nhiều nguồn liên kết, giảm khoảng trống thông tin. Tuy nhiên, Graph RAG có latency cao hơn gần 3 lần (11.8s vs. 4.2s) và thời gian build Knowledge Graph từ 45 đến 120 giây tùy kích thước tài liệu – chi phí này phát sinh từ việc gọi LLM để extract entities cho từng chunk trong quá trình indexing.

Điểm yếu quan sát được của Graph RAG trong thực nghiệm là đôi khi trộn lẫn địa điểm khi thực thể xuất hiện ở nhiều context khác nhau (ví dụ: các bí danh dùng ở Việt Nam bị gán nhầm vào giai đoạn hoạt động tại Trung Quốc). Nguyên nhân là LLM entity extraction chưa đủ chính xác với Qwen2.5:7b trên đoạn văn ngắn (500 ký tự), dẫn đến edges trong graph có thể mang quan hệ sai.

Kết luận thực nghiệm: Classic RAG phù hợp khi ưu tiên tốc độ và câu hỏi mang tính tra cứu đơn giản; Graph RAG phù hợp khi tài liệu có nhiều thực thể liên kết phức tạp và câu hỏi đòi hỏi suy luận đa bước. Với tài liệu tiểu sử có cấu trúc danh sách dày đặc như trong thực nghiệm, hybrid approach – dùng Classic RAG cho Factual query và Graph RAG cho Reasoning/Multi-hop query – có thể là hướng tối ưu cho tương lai.

III. KẾT QUẢ VÀ ĐÁNH GIÁ TỔNG HỢP

A. Tổng hợp kết quả 10 yêu cầu mở rộng

Bảng VIII trình bày đánh giá tổng hợp mười yêu cầu mở rộng theo bốn tiêu chí: Chức năng (Functionality), Chất lượng mã (Code Quality), Tài liệu (Docs) và Giao diện (UI). Thang điểm từ 1 đến 5.

Bảng VIII
ĐÁNH GIÁ TỔNG HỢP 10 YÊU CẦU MỞ RỘNG (THANG 1–5)

Câu	Tính năng	Func.	Code	Docs	UI
1	DOCX Support	5	4	4	5
2	Lưu lịch sử SQLite	5	5	4	5
3	Quản lý lịch sử	5	4	4	5
4	Tối ưu Chunk Strategy	4	4	5	4
5	Citation Tracking	5	4	4	5
6	Conversational RAG	5	5	5	4
7	Hybrid Search	5	5	4	4
8	Multi-document RAG	4	4	4	5
9	Cross-Encoder Re-ranking	5	4	4	4
10	Self-RAG + Query Rewrite	4	4	5	4
TB		4.7	4.3	4.3	4.5

B. Hiệu năng tổng thể hệ thống

Bảng IX tổng hợp các chỉ số hiệu năng đo trên máy thử nghiệm (CPU: Intel Core i5-12400, RAM: 16 GB, GPU: không dùng) với tài liệu PDF 50 trang, chunk_size=1000, top_k=5.

Bảng IX
HIỆU NĂNG HỆ THỐNG THEO CHẾ ĐỘ HOẠT ĐỘNG

Chế độ	Latency TB (s)	RAM (GB)
Classic RAG	4.2	3.1
Hybrid Search	4.8	3.2
Hybrid + Re-ranking	7.5	3.5
Self-RAG (1 iter)	9.1	3.5
Self-RAG (2 iter)	14.3	3.5
Graph RAG	11.8	4.2

Thời gian build index lần đầu: FAISS dưới 1 giây, Knowledge Graph từ 45 đến 120 giây tùy kích thước tài liệu. Sau khi build, index được lưu vào disk và nạp lại ngay lập tức ở các lần sau.

C. Ưu điểm và Hạn chế

Ưu điểm chính:

- **Hoàn toàn cục bộ và riêng tư** – không yêu cầu API key hay kết nối Internet sau khi cài đặt; dữ liệu người dùng không rời khỏi máy.
- **Kiến trúc mô-đun** – các thành phần (loader, retriever, re-ranker, LLM) dễ thay thế độc lập mà không ảnh hưởng pipeline.
- **Hỗ trợ tiếng Việt ổn định** – kết hợp paraphrase-multilingual-mpnet-base-v2 và Qwen2.5:7b cho chất lượng retrieval và sinh văn bản tiếng Việt tốt.
- **Trải nghiệm người dùng đầy đủ** – citation tracking, lịch sử hội thoại SQLite và giao diện Streamlit trực quan giúp hệ thống sẵn sàng dùng trong thực tế.

Hạn chế còn tồn tại:

- **Tốc độ với tài liệu lớn** – Self-RAG và Graph RAG có latency cao (trên 20 giây) khi tài liệu

vượt 100 trang, chưa phù hợp cho ứng dụng yêu cầu phản hồi tức thời.

- **Chưa có đánh giá tự động** – hiện tại điểm chất lượng câu trả lời được chấm thủ công; thiếu pipeline RAGAS để đo *faithfulness* và *answer relevancy* một cách tái lập.
- **Entity extraction của Graph RAG chưa ổn định** – Qwen2.5:7b đôi khi trích xuất thực thể sai trên đoạn văn ngắn, dẫn đến edges trong Knowledge Graph mang quan hệ không chính xác.
- **Giao diện chưa hỗ trợ streaming** – người dùng phải chờ toàn bộ câu trả lời được sinh xong mới thấy kết quả, gây cảm giác chậm với câu trả lời dài.

IV. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

A. Kết luận

Dự án *SmartDoc AI* đã hoàn thành thành công một hệ thống RAG hoàn chỉnh chạy cục bộ, đáp ứng toàn bộ mười yêu cầu phát triển đặt ra trong đề bài và bổ sung thêm thực nghiệm so sánh RAG với Graph RAG.

Về **kiến trúc**, hệ thống được tổ chức theo mô hình phân lớp rõ ràng: lớp giao diện (Streamlit), lớp xử lý (LangChain pipeline), lớp dữ liệu (FAISS vector store + SQLite) và lớp mô hình (Ollama/Qwen2.5:7b). Cách tổ chức này giúp từng thành phần dễ thay thế và kiểm thử độc lập.

Về **tính năng mở rộng**, việc thêm DOCX loader, hybrid search, cross-encoder re-ranking và Self-RAG đã cải thiện độ chính xác câu trả lời so với base project, đặc biệt với các câu hỏi kỹ thuật có từ khóa đặc thù mà embedding model chưa được fine-tune. Citation tracking và lưu lịch sử SQLite tăng đáng kể tính khả dụng cho người dùng thực tế.

Về **so sánh RAG vs. Graph RAG**, thực nghiệm cho thấy RAG truyền thống phù hợp với các câu hỏi tra cứu đơn giản (factual Q&A) với tốc độ phản hồi nhanh hơn khoảng X lần. Graph RAG vượt trội ở các câu hỏi đa bước suy luận (multi-hop reasoning) nhờ cấu trúc đồ thị tri thức, tuy nhiên đòi hỏi chi phí xây dựng và thời gian indexing cao hơn đáng kể. Điều này gợi ý rằng việc lựa chọn kiến trúc retrieval nên dựa vào đặc điểm của bộ tài liệu và loại câu hỏi đặc trưng của bài toán.

Về **hỗ trợ tiếng Việt**, mô hình embedding paraphrase-multilingual-mpnet-base-v2 kết hợp với Qwen2.5:7b cho thấy chất lượng trả lời tiếng Việt ổn định, đáp ứng yêu cầu

của người dùng trong bối cảnh học thuật trong nước.

Tóm lại, dự án chứng minh rằng một hệ thống RAG đầy đủ tính năng, riêng tư và miễn phí hoàn toàn có thể xây dựng từ các công cụ mã nguồn mở, mở ra tiềm năng ứng dụng rộng rãi trong giáo dục, nghiên cứu và doanh nghiệp.

B. Hướng phát triển

Dựa trên những hạn chế quan sát được trong quá trình thực nghiệm, nhóm đề xuất các hướng phát triển sau cho tháng tiếp theo:

Đánh giá tự động với RAGAS. Hiện tại nhóm đánh giá câu trả lời thủ công. Tích hợp framework RAGAS [8] sẽ cho phép đo các chỉ số chuẩn như *faithfulness*, *answer relevancy* và *context recall* một cách tự động và tái lập được.

Hỗ trợ streaming response. Qwen2.5:7b thông qua Ollama hỗ trợ token streaming. Triển khai tính năng này trên giao diện Streamlit sẽ cải thiện trải nghiệm người dùng khi câu trả lời dài, giảm cảm giác chờ đợi.

Tích hợp web search tool. Mở rộng RAG với khả năng tìm kiếm web theo thời gian thực (ví dụ qua SerpAPI hoặc Tavily) để trả lời các câu hỏi nằm ngoài nội dung tài liệu đã upload.

Triển khai lên môi trường đám mây. Đóng gói ứng dụng bằng Docker và triển khai lên nền tảng như Hugging Face Spaces hoặc Google Cloud Run để nhiều người dùng có thể truy cập mà không cần cài đặt cục bộ.

Fine-tuning embedding model cho tiếng Việt. Huấn luyện thêm embedding model trên corpus tiếng Việt chuyên ngành (pháp luật, y tế, kỹ thuật) để cải thiện chất lượng retrieval với từ vựng đặc thù.

Mở rộng Graph RAG. Xây dựng pipeline Graph RAG hoàn chỉnh tích hợp trực tiếp vào hệ thống hiện tại, cho phép người dùng chọn chế độ retrieval (Vector RAG / Hybrid / Graph RAG) tùy theo loại tài liệu và câu hỏi.

TÀI LIỆU

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS* 2020. <https://arxiv.org/abs/2005.11401>
- [2] LangChain, *LangChain Documentation*. <https://python.langchain.com/docs/introduction/>
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2019.

- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP 2019*. <https://arxiv.org/abs/1908.10084>
- [5] D. Edge et al., “From Local to Global: A Graph RAG Approach to Query-Focused Summarization,” *arXiv preprint*, 2024. <https://arxiv.org/abs/2404.16130>
- [6] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, 2009.
- [7] R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” *arXiv preprint*, 2019. <https://arxiv.org/abs/1901.04085>
- [8] S. Es et al., “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” *arXiv preprint*, 2023. <https://arxiv.org/abs/2309.15217>
- [9] Ollama, *Get up and running with large language models locally*. <https://ollama.ai/>
- [10] Streamlit, *The fastest way to build and share data apps*. <https://docs.streamlit.io/>
- [11] Alibaba Cloud, *Qwen2.5 Technical Report*, 2024. <https://qwenlm.github.io/blog/qwen2.5/>